组件命名

当注册组件（或者 prop）时，可以使用 kebab-case（短横线分隔命名）、camelCase（驼峰式命名）或 PascalCase（单词首字母大写命名）。

注册组件时键值对名字一样可以缩写成一个(es6)

组件中的data

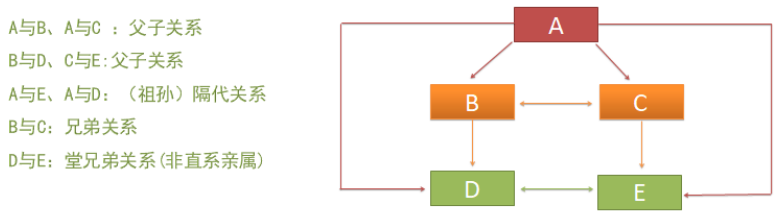
组件中的data选项必须是一个函数,返回一个数据对象。而不可以像Vue实例是一个对象呢。首先因为组件会被多次复用，而对象是引用数据类型,如果组件数据使用对象的话,的选项对象一样那么组件所有的复用都共享这些数据。

```
data(){
    return{}
}
```

组件通讯

组件数据关系:组件实例的作用域是孤立的。这意味着不能（也不应该）在子组件的模板内直接引用父组件的数据。如果在子组件中强行使用父组件的数据 ,就会报错,那么子组件如何才能获取父组件的数据呢?

组件之间的通讯又叫做组件的传值，父子组件的通讯和非父子组件的通讯。



父子组件通讯

如何进行父子组件间的通信呢？Vue官方提到

通过props向子组件传递数据

通过事件向父组件发送消息

父向子传递数据

props的值有两种方式：

方式一：字符串数组，数组中的字符串就是传递时的名称。

方式二：对象，对象可以设置传递时的类型，也可以设置默认值等。

```
<div id="app">
    <!-- 静态传值（movies）动态传值(des)
      父组件在使用子组件的时候，可以将父组件的数据绑定到使用子组件的标签上
      子组件在选项中添加一个props属性来接收数据
    -->
    <my-son :movies="movies"  :des="msg"></my-son>
</div>
<template id="son">
    <div style="border: 1px solid red;">
      <h2>这是子组件</h2>
      <h3>电影名称：{{movies}}</h3>
      <h4>{{des}}</h4>
    </div>
</template>
let MySon={
      template:'#son',
      // 子组件定义props接收数据并使用
      // 数组里存放的是自定义属性的名称
      props:['movies','des']//数组写法
  }
  new Vue({//父组件
      el:'#app',
      data:{movies:"唐人街探案3,你好，李焕英",msg:'这是父组件中的msg'},
      components:{
          // 注册的子组件
          //es6键值对名字一样可以缩写
          MySon
      }
  })
```

这是子组件

电影名称：唐人街探案3,你好，李焕英

这是父组件中的msg

props数据验证

验证支持的数据类型

String

Number

Boolean

Array

Object

Date

Function

Symbol

```
<div id="app">
   <my-son :clist="list"  mymsg="abc" :msg="msg"></my-son>
</div>
<template id="son">
          <div style="border: 1px solid red;">
            <h2>这是子组件</h2>
            <ul v-for="item in clist">
               <li>{{item}}</li>
            </ul>

            <h3>{{cmsg}}</h3>
            <h3>{{mymsg}}</h3>
            <h3>{{msg}}</h3>
          </div>
 </template>
         let MySon={
            template:'#son',
            // 子组件定义props接收数据并使用
            // 数组里存放的是自定义属性的名称
            // props:['movies','des']//数组写法
            props:{//对象的写法
               // clist:Array,//数组类型
               // clist:String
               // 一个属性，限制多个类型(建议使用数组)
               clist:[String,Array],//注意不要使用引号引起了
               // 可以使用默认值，以及一些必传值
               cmsg:{
                  /*
                     默认值触发：
                        当在template使用了props的属性，但是并没有给他传递数据时会触
发
                  */
                  type:String,
                  default:'这个是cmsg',
                  //required:true这个属性表示必须传值，但是不能和default同用
               },
               mymsg:{
                  type:String,
                  required:true,//数据是否传递，true是，false否
               },
               msg:String
            }
         }
         new Vue({//父组件
```

```
        el:'#app',
        data:{
            msg:'这是父组件中的msg',
            list:["土豆","大象","鸡肉","飞机"]
        },
        components:{
            // 注册的子组件
            MySon
        }
    })
```

这是子组件

- 土豆
- 大象
- 鸡肉
- 飞机

这个是cmsg

abc

这是父组件中的msg

子向父传递数据

什么时候需要自定义事件呢？

当子组件需要向父组件传递数据时，就要用到自定义事件了。

我们之前学习的v-on不仅仅可以用于监听DOM事件，也可以用于组件间的自定义事件。

自定义事件的流程：

在子组件中，通过$emit()来触发事件。

在父组件中，通过v-on来监听子组件事件。

```
<div id="app">
    <h3>父组件：{{des}}</h3>
    <!-- 接收子组件通过$emit发射的事件，传递参数触发父组件changs方法-->
    <my-son @biuapp="changs"></my-son>
</div>
<template id="son">
    <div style="border: 2px greenyellow solid;">
        <h2>这是子组件son</h2>
        <button type="button" @click="fn">发射</button>
    </div>
</template>

var  MySon={
     template:'#son',
     data(){//子组件的data
        return{
```

```
            // 子组件定义的msg
            msg:'我是子组件中的数据'
        }
    },
    methods:{
        fn(){//把子组件的数据给父组件
            // 发射一个事件并写的参数
            // 第一个参数：事件名，第二个参数：携带的参数
            this.$emit("biuapp",this.msg)
            console.log('已经发射')
        }
    }
}

new Vue({
    el:'#app',
    data:{
        // 父组件的des
        des:''
    },
    methods:{
        changs(value){
            this.des=value
            console.log(value)
            console.log('接收到了')
        }
    },
    components:{
      MySon
    }
})
```